라우팅 클라이언트 예제

개요

이 예제는 Delegated Routing V1 HTTP 클라이언트를 사용하여 IPFS 네트워크에서 콘텐츠 제공자를 찾고, 피어 정보를 조회하며, IPNS 레코드를 가져오는 방법을 보여줍니다. 위임된 라우팅을 통해 로컬 DHT 없이도 네트워크 정보에 접근할 수 있습니다.

이 예제를 통해 다음을 학습할 수 있습니다:

- Delegated Routing V1 사양을 사용한 라우팅 클라이언트 구현
- CID 제공자 검색 및 피어 정보 조회
- IPNS 레코드 검증 및 해석
- 스트리밍 응답 처리 및 오류 관리

주요 개념

위임된 라우팅 아키텍처

전체 동작 흐름

1. **라우팅 클라이언트**
    - HTTP 요청을 생성 (예: CID 조회, 피어 조회, IPNS 해석)
    - 요청을 라우팅 서버로 전달
2. **라우팅 서버 (Kubo/게이트웨이 등)**
    - 라우팅 클라이언트의 요청을 받아 IPFS 네트워크와 통신
    - DHT 쿼리를 IPFS DHT로 전달

    - 자체적으로 **HTTP API 처리** 및 **캐싱**을 수행

3. **IPFS DHT 네트워크**
    - 다수의 DHT 노드들이 분산되어 있음
    - DHT 쿼리에 따라 **피어 정보** 및 **콘텐츠 위치**를 반환

요청·응답의 주요 경로

- **CID 검색 / 피어 조회 / IPNS 해석**
  `라우팅 클라이언트 → 라우팅 서버`
- **DHT 쿼리 및 응답**
  `라우팅 서버 ↔ IPFS DHT (여러 노드)`
- **HTTP API 처리 및 캐싱**
  `라우팅 서버` 내부에서 실행
- **최종 결과**
  라우팅 서버가 검증·캐싱한 콘텐츠 위치, 피어 정보 등을 라우팅 클라이언트로 반환

즉 단순화하면:
`라우팅 클라이언트(요청: CID/IPNS) → 라우팅 서버(Kubo/게이트웨이, HTTP API+캐싱) → IPFS DHT 네트워크(피어 정보, 콘텐츠 위치)` → 결과 반환

주요 구성 요소

- **라우팅 클라이언트**: HTTP 요청을 통해 라우팅 정보를 조회하는 클라이언트
- **라우팅 서버**: Delegated Routing V1 API를 제공하는 서버 (예: Kubo 노드, IPFS 게이트웨이)
- **DHT 네트워크**: 실제 콘텐츠와 피어 정보가 저장된 분산 해시 테이블

전체 구현 예제

---

1. 프로젝트 설정

먼저 필요한 의존성을 포함한 Go 모듈을 생성합니다:

```
// go.mod
module routing-client-example

go 1.21

require (
github.com/ipfs/boxo v0.13.1
github.com/ipfs/go-cid v0.4.1
github.com/libp2p/go-libp2p v0.32.0
github.com/multiformats/go-multiaddr v0.12.0
)
```

2. 기본 라우팅 클라이언트 구현

```
// main.go
package main

import (
"context"
"fmt"
```

```go
"log"

"time"


"github.com/ipfs/boxo/routing/http/client"

"github.com/ipfs/go-cid"

"github.com/libp2p/go-libp2p/core/peer"

"github.com/multiformats/go-multiaddr"

)


type RoutingClient struct {

client *client.Client

}


func NewRoutingClient(serverURL string) (*RoutingClient, error) {

// Delegated Routing V1 클라이언트 생성

c, err := client.New(serverURL)

if err != nil {

return nil, fmt.Errorf("라우팅 클라이언트 생성 실패: %w", err)

}


return &RoutingClient{

client: c,

}, nil
```

```go
}


func main() {
// 공개 IPFS 게이트웨이의 라우팅 API 사용
routingClient, err := NewRoutingClient("https://cid.contact")
if err != nil {
log.Fatal(err)
}


ctx := context.Background()


// 예제 CID (IPFS 로고)
cidStr := "QmYwAPJzv5CZsnA625s3Xf2nemtYgPpHdWEz79ojWnPbdG"
c, err := cid.Decode(cidStr)
if err != nil {
log.Fatal("CID 파싱 실패:", err)
}


fmt.Printf("CID %s에 대한 제공자 검색 중...\n", cidStr)


// CID 제공자 찾기
if err := findProviders(ctx, routingClient, c); err != nil {
log.Printf("제공자 검색 실패: %v", err)
```

```
}

// 피어 정보 조회 예제
if err := findPeerInfo(ctx, routingClient); err != nil {
log.Printf("피어 정보 조회 실패: %v", err)
}

// IPNS 레코드 조회 예제
if err := resolveIPNS(ctx, routingClient); err != nil {
log.Printf("IPNS 해석 실패: %v", err)
}
}
```

3. CID 제공자 검색 구현

```
func findProviders(ctx context.Context, rc *RoutingClient, c cid.Cid) error {
fmt.Printf("\n=== CID 제공자 검색 ===\n")
// 타임아웃 설정
ctx, cancel := context.WithTimeout(ctx, 30*time.Second)
defer cancel()

// 제공자 검색 시작
providerCh, err := rc.client.FindProviders(ctx, c)
```

```
if err != nil {
return fmt.Errorf("제공자 검색 시작 실패: %w", err)
}


providerCount := 0
for provider := range providerCh {
providerCount++
fmt.Printf("제공자 #%d:\n", providerCount)
fmt.Printf(" 피어 ID: %s\n", provider.ID)
// 멀티주소 출력
if len(provider.Addrs) > 0 {
fmt.Printf(" 주소들:\n")
for _, addr := range provider.Addrs {
fmt.Printf(" %s\n", addr.String())
}
} else {
fmt.Printf(" 주소: 없음\n")
}
// 프로토콜 정보
if len(provider.Protocols) > 0 {
fmt.Printf(" 지원 프로토콜: %v\n", provider.Protocols)
}
fmt.Println()
```

```
// 최대 5개 제공자만 출력
if providerCount >= 5 {
fmt.Printf("... (더 많은 제공자가 있을 수 있습니다)\n")
break
}
}

if providerCount == 0 {
fmt.Printf("제공자를 찾을 수 없습니다.\n")
} else {
fmt.Printf("총 %d개의 제공자를 찾았습니다.\n", providerCount)
}

return nil
}
```

4. 피어 정보 조회 구현

```
func findPeerInfo(ctx context.Context, rc *RoutingClient) error {
fmt.Printf("\n=== 피어 정보 조회 ===\n")
// 예제 피어 ID (IPFS 부트스트랩 노드)
peerIDStr := "QmNnooDu7bfjPFoTZYxMNLWUQJyrVwtbZg5gBMjTezGAJN"
peerID, err := peer.Decode(peerIDStr)
```

```
if err != nil {
return fmt.Errorf("피어 ID 파싱 실패: %w", err)
}

ctx, cancel := context.WithTimeout(ctx, 15*time.Second)
defer cancel()

fmt.Printf("피어 %s 정보 조회 중...\n", peerIDStr)

// 피어 정보 조회
peerInfo, err := rc.client.FindPeer(ctx, peerID)
if err != nil {
return fmt.Errorf("피어 정보 조회 실패: %w", err)
}

fmt.Printf("피어 정보:\n")
fmt.Printf(" 피어 ID: %s\n", peerInfo.ID)
if len(peerInfo.Addrs) > 0 {
fmt.Printf(" 주소들:\n")
for _, addr := range peerInfo.Addrs {
fmt.Printf(" %s\n", addr.String())
}
} else {
```

```
fmt.Printf(" 주소: 없음\n")
}

if len(peerInfo.Protocols) > 0 {
fmt.Printf(" 지원 프로토콜: %v\n", peerInfo.Protocols)
}

return nil
}
```

5. IPNS 레코드 해석 구현

```
func resolveIPNS(ctx context.Context, rc *RoutingClient) error {
fmt.Printf("\n=== IPNS 레코드 해석 ===\n")
// 예제 IPNS 이름 (IPFS 웹사이트)
ipnsName := "ipfs.io"
fmt.Printf("IPNS 이름 '%s' 해석 중...\n", ipnsName)

ctx, cancel := context.WithTimeout(ctx, 20*time.Second)
defer cancel()

// IPNS 이름을 피어 ID로 변환
peerID, err := peer.Decode(ipnsName)
```

```
if err != nil {
// 도메인 이름인 경우 DNS를 통해 해석 시도
fmt.Printf("도메인 이름으로 인식됨: %s\n", ipnsName)
return nil // 실제 구현에서는 DNS 해석 로직 추가
}

// IPNS 레코드 조회
record, err := rc.client.GetIPNS(ctx, peerID)
if err != nil {
return fmt.Errorf("IPNS 레코드 조회 실패: %w", err)
}

fmt.Printf("IPNS 레코드 정보:\n")
fmt.Printf(" 값: %s\n", string(record.Value))
fmt.Printf(" 시퀀스: %d\n", record.Sequence)
fmt.Printf(" TTL: %v\n", record.TTL)
if record.ValidityType != nil {
fmt.Printf(" 유효성 타입: %s\n", string(record.ValidityType))
}
if record.Validity != nil {
fmt.Printf(" 유효성: %s\n", string(record.Validity))
}
```

```
return nil
}
```

6. 고급 기능 구현

스트리밍 응답 처리

```
func streamingProviderSearch(ctx context.Context, rc *RoutingClient, c cid.Cid)
fmt.Printf("\n=== 스트리밍 제공자 검색 ===\n")
providerCh, err := rc.client.FindProviders(ctx, c)
if err != nil {
return fmt.Errorf("제공자 검색 시작 실패: %w", err)
}

// 실시간으로 제공자 정보 처리
go func() {
for provider := range providerCh {
fmt.Printf("새 제공자 발견: %s\n", provider.ID)
// 제공자별 추가 처리 로직
if err := processProvider(ctx, provider); err != nil {
log.Printf("제공자 처리 실패: %v", err)
}
}
}()
```

```
return nil
}

func processProvider(ctx context.Context, provider peer.AddrInfo) error {
// 제공자 연결 테스트
for _, addr := range provider.Addrs {
if isReachable(addr) {
fmt.Printf(" 연결 가능한 주소: %s\n", addr)
return nil
}
}
fmt.Printf(" 연결 불가능한 제공자: %s\n", provider.ID)
return nil
}

func isReachable(addr multiaddr.Multiaddr) bool {
// 실제 구현에서는 연결 테스트 로직 추가
// 여기서는 간단히 TCP 주소인지 확인
protocols := addr.Protocols()
for _, p := range protocols {
if p.Code == multiaddr.P_TCP {
return true
```

```
}

}

return false

}
```

오류 처리 및 재시도 로직

```
func robustProviderSearch(ctx context.Context, rc *RoutingClient, c cid.Cid) er

const maxRetries = 3

const retryDelay = 2 * time.Second


for attempt := 1; attempt <= maxRetries; attempt++ {

fmt.Printf("제공자 검색 시도 %d/%d\n", attempt, maxRetries)

searchCtx, cancel := context.WithTimeout(ctx, 10*time.Second)

providerCh, err := rc.client.FindProviders(searchCtx, c)

if err != nil {

cancel()

if attempt == maxRetries {

return fmt.Errorf("최대 재시도 횟수 초과: %w", err)

}

fmt.Printf("시도 %d 실패, %v 후 재시도: %v\n", attempt, retryDelay, err)

time.Sleep(retryDelay)

continue

}
```

```
// 성공적으로 채널을 받았으면 처리
providerCount := 0
for provider := range providerCh {
providerCount++
fmt.Printf("제공자 %d: %s\n", providerCount, provider.ID)
}
cancel()
if providerCount > 0 {
fmt.Printf("성공: %d개 제공자 발견\n", providerCount)
return nil
}
if attempt < maxRetries {
fmt.Printf("제공자 없음, %v 후 재시도\n", retryDelay)
time.Sleep(retryDelay)
}
}

return fmt.Errorf("모든 시도에서 제공자를 찾을 수 없음")
}
```

실행 방법

1. 프로젝트 준비

```
# 프로젝트 디렉토리 생성
mkdir routing-client-example
cd routing-client-example

# Go 모듈 초기화
go mod init routing-client-example

# 의존성 설치
go mod tidy
```

2. 코드 실행

```
# 기본 예제 실행
go run main.go

# 특정 CID로 테스트
go run main.go -cid QmYwAPJzv5CZsnA625s3Xf2nemtYgPpHdWEz79ojWnPbdG

# 다른 라우팅 서버 사용
go run main.go -server https://delegated-ipfs.dev
```

3. 예상 출력

```
CID QmYwAPJzv5CZsnA625s3Xf2nemtYgPpHdWEz79ojWnPbdG에 대한 제공자 검색 중...


=== CID 제공자 검색 ===

제공자 #1:

피어 ID: 12D3KooWGC6TvWhfapngX6wvJHMYVKpDMXGb9ZnYMToDdsVkq9tA

주소들:

/ip4/147.75.83.83/tcp/4001

/ip6/2604:1380:3000:1500::1/tcp/4001

지원 프로토콜: [/ipfs/bitswap/1.2.0]


제공자 #2:

피어 ID: 12D3KooWQtpvNvUYFzAo1cRYkydgk15JrMSHp6B6oujqgYSnvsVm

주소들:

/ip4/139.178.68.217/tcp/4001

/ip6/2604:1380:2000:7a00::1/tcp/4001


총 2개의 제공자를 찾았습니다.


=== 피어 정보 조회 ===

피어 QmNnooDu7bfjPFoTZYxMNLWUQJyrVwtbZg5gBMjTezGAJN 정보 조회 중...

피어 정보:
```

```
피어 ID: QmNnooDu7bfjPFoTZYxMNLWUQJyrVwtbZg5gBMjTezGAJN

주소들:

/ip4/104.131.131.82/tcp/4001

/ip6/2604:a880:1:20::203:d001/tcp/4001

지원 프로토콜: [/ipfs/id/1.0.0, /ipfs/bitswap/1.2.0]
```

활용 사례

1. 콘텐츠 가용성 확인

```go
func checkContentAvailability(ctx context.Context, rc *RoutingClient, c cid.Cid

providerCh, err := rc.client.FindProviders(ctx, c)

if err != nil {

return false, err

}


// 첫 번째 제공자가 있으면 콘텐츠 사용 가능

select {

case provider := <-providerCh:

if provider.ID != "" {

return true, nil

}

case <-ctx.Done():
```

```
return false, ctx.Err()

case <-time.After(5 * time.Second):

return false, nil

}



return false, nil

}
```

2. 네트워크 상태 모니터링

```
func  monitorNetworkHealth(ctx context.Context, rc *RoutingClient) {

ticker := time.NewTicker(30 * time.Second)

defer  ticker.Stop()



for {

select {

case <-ticker.C:

// 알려진 피어들의 상태 확인

healthyPeers := 0

testPeers := []string{

"QmNnooDu7bfjPFoTZYxMNLWUQJyrVwtbZg5gBMjTezGAJN",

"QmaCpDMGvV2BGHeYERUEnRQAwe3N8SzbUtfsmvsqQLuvuJ",

}
```

```
for _, peerStr := range testPeers {
peerID, _ := peer.Decode(peerStr)
if _, err := rc.client.FindPeer(ctx, peerID); err == nil {
healthyPeers++
}
}

fmt.Printf("네트워크 상태: %d/%d 피어 응답\n", healthyPeers, len(testPeers))

case <-ctx.Done():
return
}
}
}
```

3. 콘텐츠 복제 전략

```
func analyzeContentDistribution(ctx context.Context, rc *RoutingClient, cids []
for _, c := range cids {
providerCh, err := rc.client.FindProviders(ctx, c)
if err != nil {
continue
```

```
}

providers := make([]peer.AddrInfo, 0)
for provider := range providerCh {
providers = append(providers, provider)
}

fmt.Printf("CID %s: %d개 제공자\n", c.String(), len(providers))
if len(providers) < 3 {
fmt.Printf(" ⚠️ 복제 부족 - 추가 복제 권장\n")
}
// 지역별 분산 분석
analyzeGeographicDistribution(providers)
}
}

func analyzeGeographicDistribution(providers []peer.AddrInfo) {
regions := make(map[string]int)
for _, provider := range providers {
for _, addr := range provider.Addrs {
// IP 주소에서 지역 추정 (실제로는 GeoIP 라이브러리 사용)
region := estimateRegion(addr)
regions[region]++
```

```
}
}
fmt.Printf(" 지역 분산: %v\n", regions)
}

func estimateRegion(addr multiaddr.Multiaddr) string {
// 간단한 지역 추정 로직
// 실제 구현에서는 GeoIP 데이터베이스 사용
return "Unknown"
}
```

모범 사례

1. 타임아웃 관리

```
// 적절한 타임아웃 설정
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

// 작업별 다른 타임아웃
providerCtx, _ := context.WithTimeout(ctx, 15*time.Second)
peerCtx, _ := context.WithTimeout(ctx, 10*time.Second)
```

2. 오류 처리

```
func handleRoutingError(err error) {
switch {
case errors.Is(err, context.DeadlineExceeded):
log.Printf("요청 타임아웃: %v", err)
case errors.Is(err, context.Canceled):
log.Printf("요청 취소됨: %v", err)
default:
log.Printf("라우팅 오류: %v", err)
}
}
```

3. 리소스 관리

```
func (rc *RoutingClient) Close() error {
// 클라이언트 리소스 정리
return rc.client.Close()
}

// 사용 후 정리
defer routingClient.Close()
```

문제 해결

일반적인 문제들

1. **제공자를 찾을 수 없음**

- CID가 네트워크에 공개되지 않았을 수 있음
- 라우팅 서버가 해당 콘텐츠를 인덱싱하지 않았을 수 있음

2. **타임아웃 오류**

- 네트워크 연결 상태 확인
- 타임아웃 값 조정
- 다른 라우팅 서버 시도

3. **피어 연결 실패**

- 방화벽 설정 확인
- NAT 환경에서의 연결 제한
- 피어의 주소 정보가 오래되었을 수 있음

디버깅 팁

```
// 상세한 로깅 활성화
import "log/slog"

logger := slog.New(slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{
Level: slog.LevelDebug,
}))
```

```
// 요청/응답 로깅

func logRequest(method, url string) {

logger.Debug("라우팅 요청", "method", method, "url", url)

}
```

이 예제를 통해 Delegated Routing V1을 사용한 효율적인 IPFS 네트워크 상호작용을 구현할 수 있습니다. 로컬 DHT 없이도 네트워크 정보에 접근할 수 있어 경량 클라이언트나 제한된 환경에서 특히 유용합니다.